

UE Algorithmique et Programmation Scientifique :

Projets de TP en langage Python

UFR de Chimie – Université Clermont Auvergne

Année 2024–2025

Ce fascicule contient les énoncés des projets de TP de Python qui vous sont proposés. Vous ne devez en choisir qu'*un seul*, sur lequel vous travaillerez en binôme pendant les six séances de TP d'algorithmique.

Au terme du semestre, vous rendrez un fichier contenant votre programme, ainsi que les éventuels programmes d'analyses séparées et les éventuels fichiers d'entrée qui devront permettre à votre enseignant de faire fonctionner correctement votre code. Vous rédigerez également un rapport expliquant votre programme, les difficultés rencontrées, comment vous les avez surmontées... Le rapport contiendra ensuite une présentation scientifique de résultats produits avec votre programme.

TP 1 – Cinétique chimique de systèmes complexes	page 2
TP 2 – Simulation d'une dynamique gravitationnelle	page 7
TP 3 – Étude d'une transition de phase	page 11
TP 4 – Simulation de croissance cristalline	page 15
TP 5 – Simulation de la structure d'un fluide	page 19

TP 1 – Cinétique chimique de systèmes complexes

1 Rappels

La vitesse d'une réaction chimique dépend du mécanisme impliqué. Un mécanisme élémentaire résulte d'une collision / réaction réelle entre les réactifs :

- ordre 1 : $A \rightarrow$ produits
- ordre 2 : $A + B \rightarrow$ produits

On considère que l'ordre 3 est peu probable pour un mécanisme élémentaire car il implique une collision simultanée de trois molécules. Il est possible de résoudre analytiquement les équations différentielles qui décrivent ces réactions et de déduire l'évolution des différentes quantités de matière en fonction du temps.

L'évolution temporelle d'un système chimique plus complexe est décrite par un système de plusieurs équations différentielles couplées (chacune décrivant un mécanisme élémentaire). La résolution mathématique analytique est généralement impossible, et on doit alors faire des approximations. On peut également utiliser une résolution numérique approchée :

- en utilisant un pas de temps fixe (approche mathématique classique),
- en utilisant un pas de temps adaptatif (ce TP).

La deuxième méthode utilise une approche probabiliste, c'est-à-dire qu'on tire au hasard les réactions susceptibles d'avoir lieu suivant leur probabilité. Cette approche, appelée Monte Carlo Cinétique (en anglais KMC pour *Kinetic Monte Carlo*), a pour avantages de pouvoir gérer en même temps des réactions ayant des constantes de vitesse très différents, elle reste simple pour des systèmes de taille quelconque, et elle est plus intuitive pour le chimiste.

2 L'équation maîtresse

Soit un ensemble de M réactions chimiques, chacune étant définie par sa constante de vitesse k . Pour une réaction i , on définit c_i ¹ tel que $c_i = k_i$ pour une réaction monomoléculaire (ordre 1) et $c_i = k_i/V$ pour une réaction bimoléculaire (ordre 2). On définit également h_i comme étant le nombre de façons possibles d'effectuer la réaction i :

1. Attention : c_i n'est bien sûr pas une concentration !

- pour une réaction de type 0 (ordre 1 : $A \rightarrow \dots$), on a $h_i = X_A$
- pour une réaction de type 1 (ordre 2 : $2A \rightarrow \dots$), on a $h_i = \frac{X_A(X_A-1)}{2}$
- pour une réaction de type 2 (ordre 2 : $A + B \rightarrow \dots$), on a $h_i = X_A X_B$

où X_j est la population actuelle de l'espèce j . On démontre que la probabilité $\mathcal{P}(\tau, m)$ à un temps t que la réaction m ait lieu (et pas une autre) pendant l'intervalle de temps $[t, t + \tau]$ est :

$$\mathcal{P}(\tau, m) = h_m c_m \exp \left(-\tau \sum_{i=1}^M h_i c_i \right) \quad (1)$$

Ceci est l'équation maîtresse de notre système : elle décrit l'évolution temporelle des populations de toutes les espèces chimiques.

La résolution de l'équation maîtresse se fait de manière stochastique : on sélectionne au hasard quelle réaction a lieu et combien de temps elle prend, en considérant des poids statistiques qui respectent l'équation maîtresse. Plus on considère un grand nombre de réactions, plus la statistique est bonne et plus on approche la solution de l'équation maîtresse avec précision.

3 Algorithme

Le grand avantage de cette méthode est qu'on peut *complexifier le système à volonté* en augmentant le nombre d'espèces chimiques et de réactions élémentaires : l'algorithme conserve sa simplicité. À volume constant, c_i est calculé une fois pour toutes alors que h_i doit être mis à jour à chaque fois que les populations changent. On définit le produit :

$$a_i = h_i c_i \quad (2)$$

qui sera la grandeur caractéristique à chaque instant de chaque réaction. a_i est proche de la définition de la vitesse de la réaction i définie en cours de cinétique de S3. Pour chaque réaction i , on calcule donc a_i ainsi que la somme $a = \sum_j a_j$.

On initialise le temps arbitrairement à $t = 0$, puis on effectue les opérations suivantes jusqu'à ce que t parvienne à une valeur fixée par l'utilisateur :

1) On génère deux nombres aléatoires r_1 et r_2 dans l'intervalle $[0, 1[$.

2) Le premier sert à calculer le temps τ qui va s'écouler avant la prochaine réaction :

$$\tau = \frac{1}{a} \ln \left(\frac{1}{r_1} \right) \quad (3)$$

3) Le deuxième nombre aléatoire sert à déterminer laquelle des réactions chimiques va avoir lieu pendant ce temps τ . La réaction m est sélectionnée telle que :

$$\sum_{i=1}^{m-1} a_i < r_2 a \leq \sum_{i=1}^m a_i \quad (4)$$

Il s'agit ici de choisir l'une des M réactions, chacune d'elle étant pondérée par son poids a_i . Pour en tirer une au hasard en tenant compte de ce poids, on voit bien que chaque réaction a une probabilité a_i/a . L'équation ci-dessus correspond à ce tirage, vu en cours.

4) Une fois la réaction sélectionnée, on met à jour les populations des réactifs et produits impliqués dans celle-ci, et on avance le temps de τ .

4 Déroulement du projet

Le premier objectif de ce projet est tout d'abord l'organisation des données : on a d'un côté des espèces chimiques, et de l'autre des réactions. Un certain nombre de tableaux vous seront nécessaires pour stocker les informations nécessaires. Notez bien que le but est d'avoir un programme général, c'est-à-dire un nombre arbitraire d'espèces et de réactions.

Votre deuxième préoccupation concerne les unités : les populations des espèces sont des nombres entiers, mais vous voulez des concentrations. On a bien sûr la concentration $C_i = n_i/V$ avec n_i en moles. Ainsi, plus vous simulez de molécules, plus vous représentez un grand volume, et plus la statistique sera bonne². Il est conseillé d'utiliser en entrée les concentrations initiales C_i^0 et le nombre total initial de molécules (toutes espèces confondues), et de déduire les populations initiales.

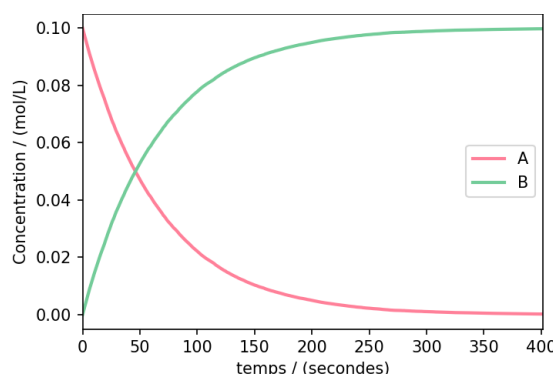


FIGURE 1 – Une simple réaction d'ordre 1 : $A \rightarrow B$. On retrouve bien la relation analytique $C_A = C_A^0 \exp(-kt)$.

La figure ci-dessus montre l'évolution des concentrations en fonction du temps dans le cas simpliste d'une seule réaction d'ordre 1. Pour pouvoir tracer cette évolution, votre programme devra

2. et plus le calcul sera lent...

écrire dans un fichier les concentrations à intervalles de temps réguliers (choisis par l'utilisateur).

Il est assez difficile de permettre à l'utilisateur d'imposer toutes les données du système. Le programme `kmcvars.py` fourni sur l'ENT vous permettra de lire facilement un fichier contenant les différentes espèces, leurs concentrations initiales ainsi que la liste des réactions. Vous pourrez ainsi tester des systèmes plus complexes. La figure suivante montre l'évolution d'un équilibre estérification/hydrolyse (deux réactions d'ordre 2) :

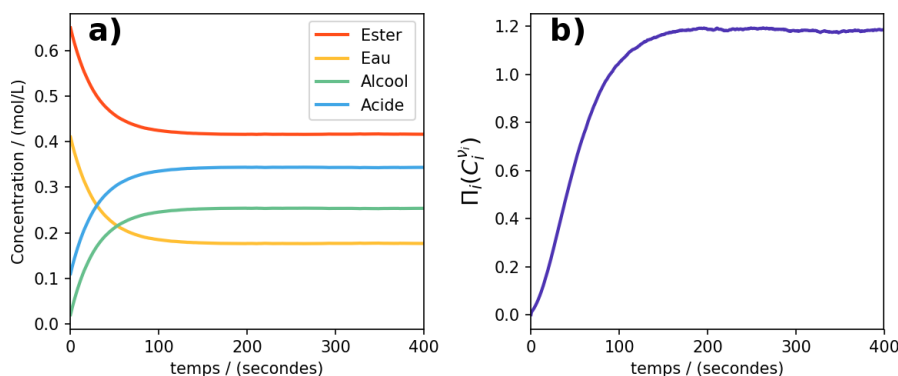


FIGURE 2 – a) Évolution temporelle des concentrations des différentes espèces. On reconnaît la décroissance hyperbolique caractéristique d'une réaction d'ordre 2. b) Lorsque $Q = \prod_i C_i^{\nu_i}$ devient constant, on est à l'équilibre.

Vous pouvez ensuite vous intéresser à des systèmes ne possédant pas de solution analytique, comme les réactions oscillantes du modèle de Lotka-Volterra :

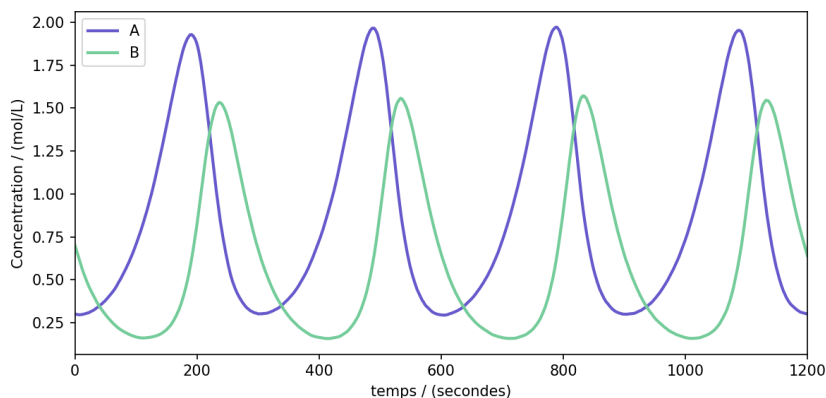
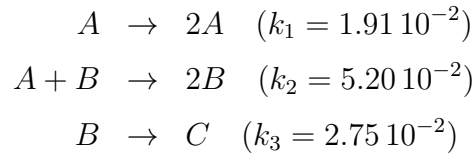


FIGURE 3 – Oscillations des concentrations en A et B en fonction du temps.

Ce modèle fut initialement proposé pour l'étude de populations biologiques, A et B représentant respectivement les proies (se multipliant naturellement) et les prédateurs (se multipliant en consommant les proies). Cette interprétation du modèle se lit facilement dans les réactions suivantes :



Les constantes de vitesse proposées ici permettent de créer les oscillations. Vous pouvez explorer d'autres valeurs pour étudier les différents types de comportements et les interpréter. Les systèmes oscillants peuvent facilement devenir chaotiques, avec une sensibilité aux conditions initiales. Pour étudier la stabilité des oscillations, on trace les concentrations en A et B l'une en fonction de l'autre :

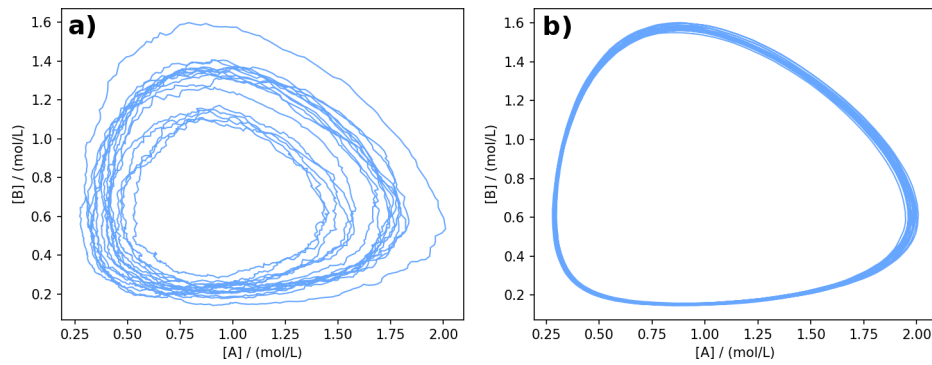


FIGURE 4 – Une meilleure façon de représenter la régularité des oscillations : a) avec $N^0 = 1000$ b) avec $N^0 = 100\,000$ (moins de 5 min de calcul).

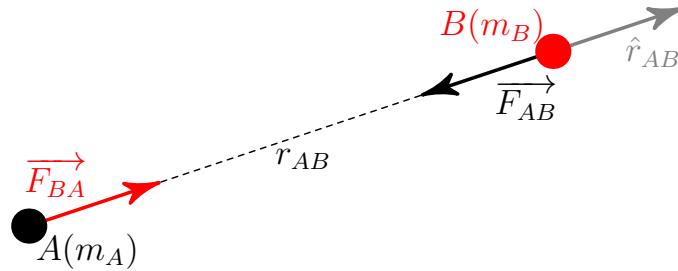
TP 2 – Simulation d’une dynamique gravitationnelle

1 Rappels sur les forces de gravitation

Soient deux particules A et B de masses respectives m_A et m_B . Il s’exerce entre ces deux masses une attraction causée par la gravitation, dont l’énergie ne dépend que des masses des particules et de la distance r_{AB} les séparant :

$$U_{AB} = -\mathcal{G} \frac{m_A m_B}{r_{AB}} \quad (1)$$

où $\mathcal{G}$ est une constante positive. Une force entre les particules dérive de cette énergie : comme l’énergie est toujours négative (les masses et distances sont positives), il s’agit d’une force d’attraction qui tend à rapprocher les deux particules. Le schéma ci-dessous illustre le phénomène :



Une force est une quantité vectorielle : $\vec{F}_{BA}$ est la force exercée par B sur A , elle est portée par la droite (AB) , définie par le vecteur unitaire $\hat{r}_{AB} = \frac{\vec{r}_{AB}}{r_{AB}}$. L’intensité de la force s’exerçant sur chaque particule est donc la dérivée de l’énergie par rapport à la distance :

$$F_{AB} = F_{BA} = \mathcal{G} \frac{m_A m_B}{r_{AB}^2} \quad (2)$$

Les deux forces ont la même intensité mais des directions opposées, de sorte que la quantité de mouvement totale se conserve (principe d’action-réaction). Le principe fondamental de la dynamique relie le total des forces s’exerçant sur une particule à son accélération :

$$m_i \vec{a}_i = \vec{F}_i = \sum_j \vec{F}_{ji} \quad (3)$$

où $\vec{a}_i$ est l’accélération de la particule i et $\vec{F}_i$ est la somme vectorielle de toutes les forces s’exerçant sur i .

2 Algorithme

L'accélération d'un objet est la dérivée de sa vitesse qui est elle-même la dérivée de sa position, par rapport au temps. Ainsi, le calcul des forces d'interactions gravitationnelles entre toutes les particules de notre système doit pouvoir conduire aux positions et aux vitesses. Les particules étant toutes en interaction, la résolution analytique d'un tel système est très souvent impossible : on se tourne alors vers une méthode numérique, facile à programmer.

Il existe plusieurs méthodes numériques pour résoudre les équations du mouvement, plus ou moins précises et stables. On utilise ici l'algorithme de Verlet à un pas. Pour cela, on fait le choix d'un pas de temps τ dit *pas d'intégration* : chaque nouvelle configuration calculée sera distante d'un temps τ de la précédente. On part d'une configuration initiale connue (positions et vitesses de chaque particule du système, temps).

À partir d'une configuration à un temps t , on déduit la configuration au temps $t + \tau$ de la façon suivante. À chaque pas de temps, on calcule successivement :

- les accélérations $\vec{a}_0$ (à partir des forces : équation 3)
- les nouvelles positions : $\vec{r} = \vec{r} + \tau \vec{v} + \frac{\tau^2}{2} \vec{a}_0$
- les accélérations mises à jour : $\vec{a}_1$
- les nouvelles vitesses : $\vec{v} = \vec{v} + \frac{\tau}{2} (\vec{a}_0 + \vec{a}_1)$

Après ces étapes de calcul, on met à jour le temps en lui ajoutant τ .

On réitère ces étapes jusqu'à ce que le temps ait atteint une valeur indiquée par l'utilisateur. Pendant la dynamique, on sauve certaines configurations dans un fichier à intervalles de temps fixes afin de reproduire graphiquement les trajectoires et/ou faire des analyses ultérieures.

3 Déroulement du projet

La progression de ce projet est assez linéaire : vous devez tout d'abord créer un système de départ composé d'un ensemble de corps célestes (masses, vitesses et positions initiales). Vous pourrez alors tester l'intégration des forces en générant les coordonnées / vitesses au cours du temps. Il est indispensable de maîtriser les unités dans ce projet. Le système international utilise des masses en kilogrammes, des distances en mètres et des temps en secondes. La constante gravitationnelle est $\mathcal{G} = 6.67408 \cdot 10^{-11} \text{ J kg}^{-2} \text{ m}^3$. La visualisation en 3D n'étant pas très aisée, il vous est conseillé de travailler en 2D.

Le choix du pas de temps τ dépend de la vitesse maximale des objets simulés, et donc du système choisi : s'il est trop grand la résolution numérique devient vite fautive (elle n'est vraie que si $\tau \rightarrow 0$), mais s'il est trop petit il faudra un temps de calcul trop important pour obtenir des trajectoires intéressantes.

Il est fortement conseillé pour ce projet de sauvegarder seulement quelques positions dans un fichier (avec un intervalle de temps choisi), et de faire des analyses / visualisations après coup en créant d'autres programmes dits de *post-traitement* : cela vous évitera de recalculer la trajectoire à chaque fois, ce qui peut prendre du temps.

Contrairement à ce qu'on pourrait croire, une visualisation esthétiquement plaisante n'est pas évidente en gravitation, car les différences d'échelle sont grandes. Vous pouvez par exemple commencer avec un système Soleil-Terre-Lune et tenter de tracer les positions. L'animation est primordiale ici, car c'est le mouvement des planètes les unes par rapport aux autres qui est important : créez un programme qui transforme une trajectoire en une série d'images à animer.

Le système le plus simple est constitué de deux corps seulement, l'un étant beaucoup plus massif que l'autre. À l'aide d'un programme d'analyse, vous pouvez vérifier numériquement les lois de Kepler, ou bien retrouver la *vitesse de libération* représentée sur la figure suivante.

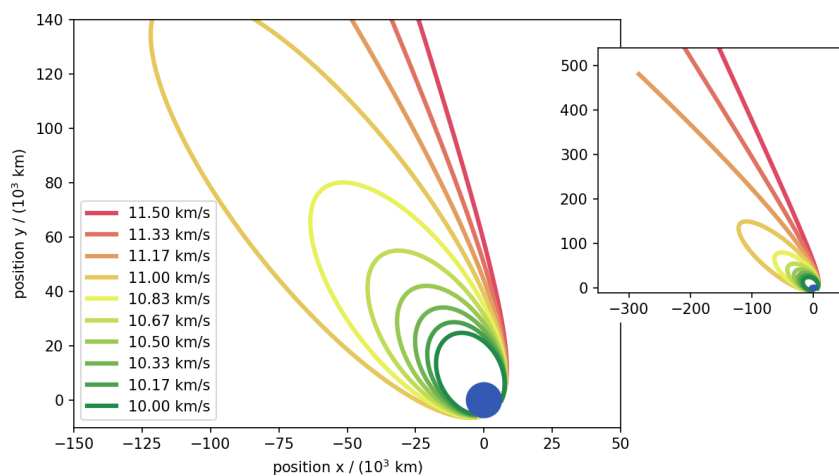


FIGURE 1 – Trajectoires d'une fusée lancée de la surface de la terre, avec différentes vitesses initiales. Chaque simulation s'arrête s'il y a collision ou si on atteint un certain nombre de pas de temps.

Vous vous apercevrez qu'il n'est pas si facile de trouver des conditions initiales formant des orbites stables. Le problème à trois corps est intéressant car il montre l'instabilité naturelle de la gravitation, alors qu'on aurait tendance à croire le contraire tant les trajectoires des étoiles/planètes sont apparemment stables. Pour mettre ce phénomène en évidence, il suffit de partir de trois objets ayant *des masses proches* (ici, trois «soleils» de masse 2×10^{30} kg). Malgré un départ d'une symétrie parfaite, il y aura forcément apparition d'un comportement erratique et imprévisible. Dans la figure suivante, on a utilisé des vitesses initiales égales et symétriques

afin de retarder le plus possible l'apparition du chaos.

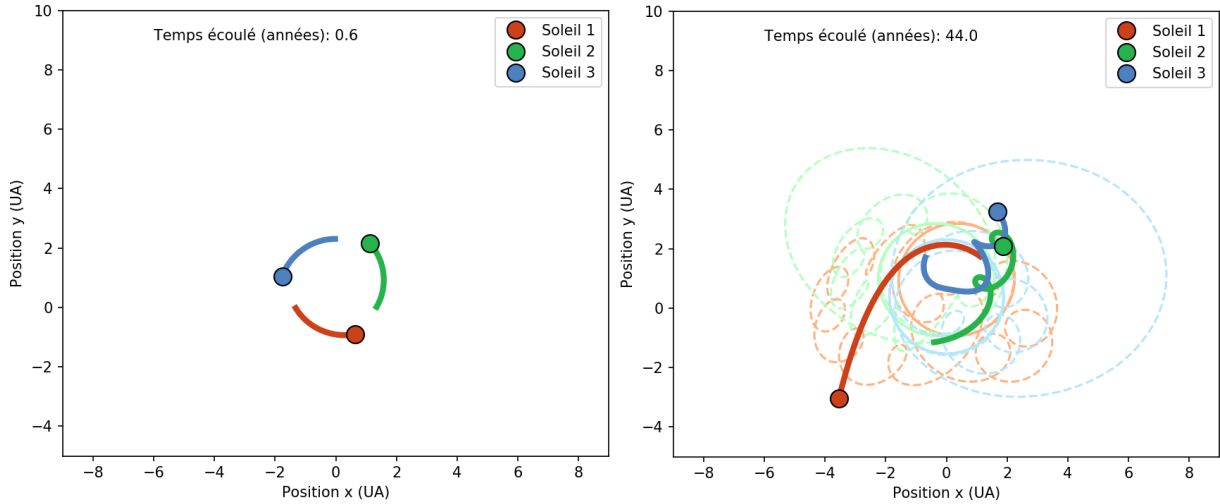


FIGURE 2 – À gauche : une symétrie de départ parfaite, avec des trajectoires circulaires bien stables. À droite : la suite, si on attend un petit moment (voir <https://youtu.be/Kndt93Q2gXY>).

Une telle instabilité est causée par la *sensibilité aux conditions initiales*, une des caractéristiques du chaos mathématique. Étudiez différentes trajectoires suivies par les objets si vous changez de façon infinitésimale l'une des positions / vitesses de la configuration initiale. Vous pouvez aussi analyser les différences entre deux trajectoires ainsi générées, comme dans la figure suivante.

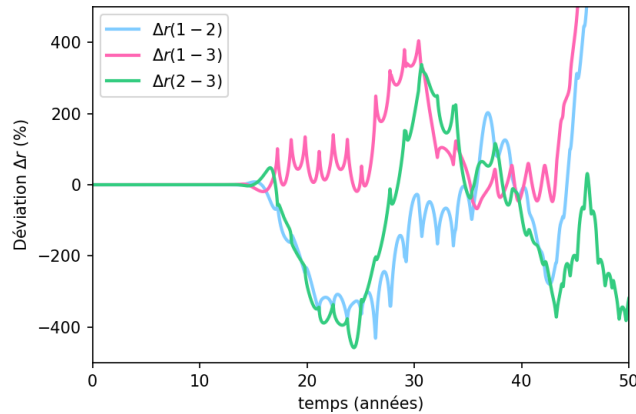


FIGURE 3 – Déviation des distances entre les trois corps dans le cas de deux trajectoires très proches initialement. La différence est quasi nulle au départ (mêmes trajectoires), puis l'évolution devient imprévisible (plus de 100% de déviation).

TP 3 – Étude d’une transition de phase

1 Principe

Le modèle d’Ising est l’un des modèles les plus simples de la physique permettant d’observer une transition de phase, phénomène universel impliquant un changement brutal d’un matériau dont l’origine se situe à l’échelle moléculaire. La transition observée est celle dans un métal entre une phase ferromagnétique (le solide crée son propre champ magnétique) et une phase paramagnétique (l’aimantation n’apparaît que si le solide est placé dans un champ extérieur). Cette transition a lieu à une température donnée appelée température de Curie T_c (pour le fer $T_c = 1043$ K).

L’aimantation provient des moments magnétiques de spin des électrons, déterminés par le nombre quantique s pouvant prendre les valeurs $-1/2$ et $+1/2$. Deux atomes voisins dans un cristal peuvent donc avoir leurs spins parallèles (mêmes valeurs de s) ou antiparallèles (valeurs différentes). Dans un solide ferromagnétique, les spins ont tendance à adopter des configurations parallèles, ce qui aura pour effet une magnétisation non nulle (création d’un champ magnétique). Deux spins parallèles sont donc plus favorables énergétiquement, ce qui peut se réduire à la relation suivante :

$$U_{ab} = -JS_aS_b \quad (1)$$

où U_{ab} est l’énergie potentielle entre les atomes voisins a et b , J est une constante positive et S_i est le spin de l’atome i ($\pm 1/2$). On voit bien que si S_a et S_b ont le même signe (spins parallèles), l’énergie est négative (plus stable), contrairement au cas antiparallèle.

Le modèle d’Ising à deux dimensions considère une grille de N sites, chacun défini par son spin S_i . En calculant l’énergie totale d’une configuration initiale, on fait ensuite évoluer le système jusqu’à l’équilibre thermodynamique. La température joue ici un rôle fondamental, car c’est elle qui apporte l’énergie nécessaire à l’évolution du système.

2 Algorithme

On considère une grille carrée de $N = L_x \times L_y$ cases représentant chacune un atome. Les voisins d’une case donnée sont représentés sur le schéma page suivante. Pour éviter des effets de bord, on applique les conditions aux limites périodiques (voir le chapitre 6 du cours).

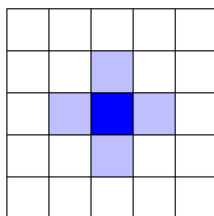


FIGURE 1 – On considère ici comme voisines de la cellule centrale les 4 cellules qui partagent un côté avec elle.

On part d'une densité donnée de spins positifs, les autres cases ayant un spin négatif. Le but est de faire évoluer cette configuration initiale jusqu'à un état d'équilibre représentatif de la température imposée. Pour cela, on utilise un algorithme très simple :

- 1) On choisit au hasard un atome. On va voir s'il change de spin ou pas. . .
- 2) On calcule deux énergies : avant et après le changement du spin. On en déduit la variation d'énergie ΔU causée par l'éventuel changement
- 3) La probabilité d'accepter ce changement est $\mathcal{P} = \exp(-\frac{\Delta U}{kT})$ où k est la constante de Boltzmann. On en déduit si le changement de spin tenté est accepté par un tirage de probabilité simple vu en cours (si non, on ne fait rien).

On répète ces étapes un (très) grand nombre de fois : le système évolue vers un état d'équilibre, avec une densité de spins homogène sur toute la grille. On définit un *mouvement* comme étant une tentative comme décrite précédemment. Un *cycle* est une succession de N mouvements. La figure suivante montre un exemple d'évolution.

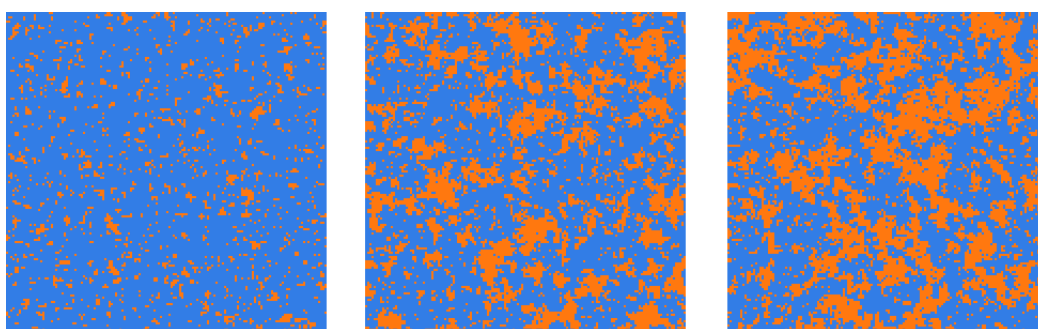


FIGURE 2 – Exemple d'évolution d'une configuration proche de T_c , en partant d'une densité initiale $\rho^0(-1/2) = 1$ (tous les spins à $-1/2$, en bleu). De gauche à droite : après 5, 100 et 200 cycles.

La composition initiale de la grille n'est pas censée avoir d'influence sur l'état d'équilibre, car c'est seulement la température qui impose l'évolution du système (elle autorise ou pas les mouvements tentés).

3 Déroulement du projet

Dans un premier temps, vous devez mettre au point l'algorithme décrit dans la section précédente. Réfléchissez à l'organisation des données et aux paramètres permettant de gérer une simulation (nombre de cycles...). On utilise ici des unités *réduites* (adimensionnelles) : on pose $J/k = 1$, ce qui simplifie les calculs et les analyses.

Une question cruciale dans ce projet pour une configuration donnée est : est-on à l'équilibre ? En effet, vous constaterez vite que l'évolution d'une même configuration peut varier dès lors qu'on utilise une graine aléatoire différente (voir <https://youtu.be/YGOFbxakQ0k>). Vous devez donc distinguer une phase d'*équibration*, pendant laquelle on considère que le système n'est pas encore à l'équilibre, et une phase d'*acquisition*, pendant laquelle on pourra analyser le système et en tirer des valeurs moyennes³.

La magnétisation (réduite) du système correspond à la moyenne des spins des N cases :

$$M = \frac{2}{N} \sum_i S_i \quad (2)$$

Au delà de la température de Curie, l'orientation aléatoire des spins rend M nulle, ce qui n'est pas le cas à basse température. Ici on multiplie par 2 pour obtenir une magnétisation entre -1 et 1 . On obtient une valeur instantanée de M pour une configuration donnée, mais on fait ensuite la moyenne sur toutes les valeurs calculées à chaque cycle pendant l'acquisition. La fluctuation $\sigma(M)$ (écart-type) est également intéressante à calculer :

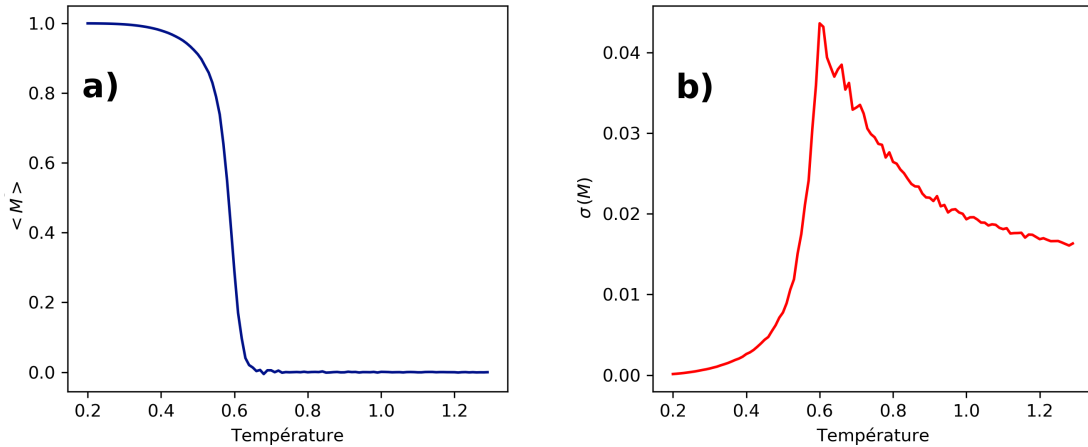


FIGURE 3 – Évolution de a) la magnétisation moyenne $\langle M \rangle$ et b) sa fluctuation $\sigma(M)$ en fonction de la température. À chaque température, on est parti de 100% de spins $S_i = +1/2$.

Lorsque la température est faible, $|M|$ est maximale car tous les spins sont orientés dans la même direction. Lorsqu'on augmente la température, la barrière d'énergie U_{ab} peut être franchie de plus en plus facilement, ce qui autorise un mélange des différents spins (du coup, M

3. Le même principe se retrouve pour toute mesure expérimentale : on attend la stabilisation avant de noter la grandeur mesurée.

diminue). À partir de T_c , la température est suffisante pour que la barrière d'énergie soit négligeable, d'où une répartition aléatoire des spins.

Les grilles de spins à deux états se prêtent à une visualisation graphique. Vous pouvez conserver l'«âge» d'un spin et inclure cette information dans votre visuel :

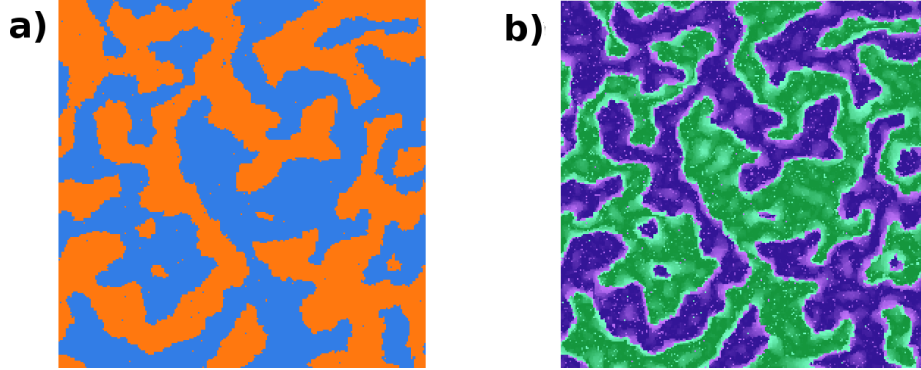


FIGURE 4 – Deux façons d'afficher une même configuration : a) deux couleurs suivant la valeur du spin b) couleurs des spins suivant le nombre de cycles passés dans le même état.

Il y a beaucoup de comportements complexes à explorer, avec parfois un temps très long pour converger. Commencer avec deux phases distinctes crée une interface quasi rectiligne, métastable :

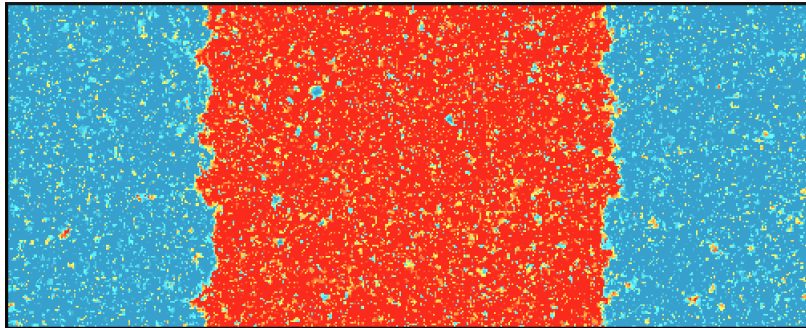


FIGURE 5 – Équilibre entre deux phases sur une grille de 500×200 à $T < T_c$: le centre est riche en $+1/2$ tandis que les côtés sont riches en $-1/2$. Il faudrait un temps très long pour converger vers l'équilibre (une seule phase).

Vous pouvez également tracer le diagramme de phases en constatant qu'à basse température, l'état d'équilibre dépend de la densité initiale de spins ($M_{eq} < 0$ si $\rho^0(-1/2) > 0.5$ et $M_{eq} > 0$ si $\rho^0(+1/2) > 0.5$). En choisissant deux configurations de départ distinctes pour chaque température, vous obtenez un diagramme qui rappelle l'équilibre liquide-vapeur du corps pur, avec son point critique à T_c .

TP 4 – Simulation de croissance cristalline

1 Principe

La formation d'un cristal peut se faire en laissant croître un solide grâce au contact avec une solution saturée. Lorsqu'on laisse évaporer le solvant, le solide précipite ; si le processus suffisamment lent, les ions peuvent se réorganiser autour d'un solide existant (point de nucléation) pour former des monocristaux de taille appréciable.

On se propose de modéliser ce phénomène avec un modèle simpliste : une grille carrée contenant des particules mobiles, lesquelles peuvent se fixer si elles se retrouvent voisines d'une particule fixe. La particule mobile se fixe alors en augmentant ainsi le solide, et peut servir de support à d'autres particules mobiles pour faire grandir le cristal.

L'utilisation d'une grille 2D au lieu de 3D permet d'être plus rapide, d'autant plus que la discrétisation de l'espace rend facile l'identification des voisins (chapitre 6 du cours).

Les résultats produits avec ce modèle très simple sont toutefois variés et intéressants, et il est très difficile de les obtenir par des modèles analytiques.

2 Algorithme

On utilisera des règles de voisinage à 4 voisins, mais il est aussi possible de travailler à 8 voisins ou même 6 pour représenter une structure hexagonale.

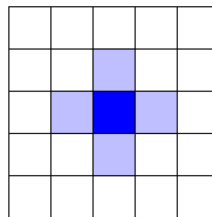


FIGURE 1 – On considère ici comme voisines de la cellule centrale les 4 cellules qui partagent un côté avec elle.

On ne considère pas d'interactions entre les particules mobiles : elles auront donc des mouvements indépendants les unes des autres. La conséquence est qu'il peut se trouver plus d'une particule aux mêmes coordonnées : il faudra donc utiliser une liste de particules (chacune étant repérée par ses coordonnées), et on construira une grille (tableau) seulement lorsqu'on voudra

tester la proximité d'une particule fixe.

À chaque itération, on fait bouger chaque particule mobile d'une case dans l'une des directions voisines (au hasard). Cela représente un mouvement de diffusion dans un solvant (marche au hasard), qui se rencontre aussi lors de l'agrégation de plus grosses entités que les molécules.

On doit aussi garder une liste des particules fixes : celles qui représentent le solide. Il faut au moins une particule fixe autour de laquelle le cristal peut s'amorcer, mais pour modéliser une surface on préfère en mettre au début une ligne pleine (dernière ligne de la grille). Les particules mobiles quant à elles sont initialisées avec des coordonnées choisies aléatoirement dans le système.

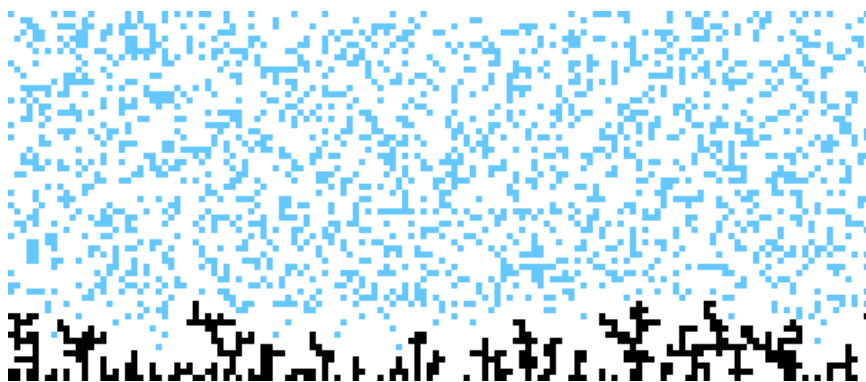


FIGURE 2 – Surface (en noir) en train de grandir grâce aux particules mobiles (en bleu) qui se déposent progressivement.

Que doit-il se passer si une particule quitte le domaine $L_x \times L_y$ imposé ? Une erreur se produirait lors de la construction de la grille car on tomberait alors en dehors des limites du tableau. On impose sur les côtes des *conditions aux limites périodiques* introduites au chapitre 6 : une particule qui quitte le système par la gauche se retrouve à droite, et inversement.

En bas il n'y a pas besoin de s'inquiéter de cela car les particules fixes initiales empêchent toute traversée. En haut, il suffit de « fermer » le système en interdisant de traverser : si la case d'arrivée d'une particule dépasse la limite, on ne fait simplement pas le mouvement et la particule reste sur place.

Chaque itération contient une phase de diffusion des particules mobiles, suivie d'une phase de fixation des particules, en testant le voisinage de chaque particule mobile à l'aide de grilles : si une particule mobile est voisine d'une particule fixe, on la change de liste pour la prochaine itération.

On répète les itérations (en sauvent régulièrement des analyses ou une image du système), jusqu'à ce qu'aucune particule mobile ne subsiste.

3 Dérroulement du projet

La réalisation de ce projet est assez simple, mais il faut bien organiser le travail avec des fonctions qui effectuent les opérations élémentaires nécessaires au modèle. Commencez par inventorier les variables nécessaires pour construire un système initial. Ensuite, listez chaque action élémentaire pour en faire une fonction.

Il faut rapidement visualiser votre système, en utilisant un graphique matplotlib (affichage d'une image) ou en créant l'image vous-même avec le module image de matplotlib. Une fois que vous obtenez des motifs corrects, vous pouvez tester différentes graines aléatoires pour constater que la forme du motif cristallin n'est jamais la même. Vous pouvez aussi tester différentes densités initiales en particules mobiles.

Pour rendre le modèle (un peu) plus réaliste, on peut autoriser les particules fixes à se déplacer à la condition que leur mouvement soit énergétiquement favorable, c'est-à-dire ici si elles *augmentent* leur nombre de voisins fixes. Une étape de diffusion de surface correspond donc à une tentative de mouvement aléatoire de chaque particule fixe, mais il ne se fait que s'il est conforme. En réalité ce réarrangement de surface est courant et permet d'obtenir des monocristaux de grande taille. Il ne peut pas se faire à l'échelle atomique si on construit un solide covalent, mais il est très courant pour les métaux et les cristaux ioniques.

La figure suivante montre quelques exemples de motifs obtenues après 85 itérations sur une grille de 60 lignes et 140 colonnes. On notera qu'avec la diffusion de surface, la formation de plus gros agrégats est favorisée.

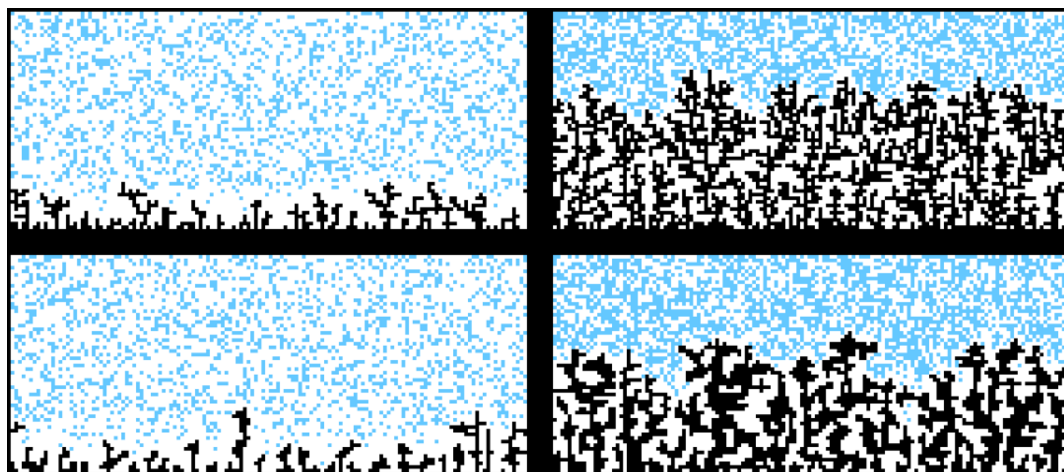


FIGURE 3 – Quatre configurations utilisant une densité initiale de particules fixes de 0.3 (les deux de gauche) et 0.7 (les deux de droite). Les simulations du haut sont faites sans diffusion de surface, alors que les deux du bas font 5 étapes de diffusion de surface pour chaque itération (voir la vidéo).

Vous pouvez aussi orienter votre projet avec une étude plus statistique en effectuant plusieurs

simulations avec les mêmes paramètres initiaux : la chaîne des nombres aléatoire n'étant pas la même, on aura des simulations différentes. En faisant des moyennes sur toutes ces simulations, on peut étudier l'évolution moyenne du nombre de particules fixes en fonction du temps, et la comparer avec une simple simulation. On constatera qu'il faut beaucoup de simulations pour obtenir un comportement moyen stable, et que chaque simulation diffère souvent beaucoup du comportement moyen.

TP 5 – Simulation de la structure d'un fluide

1 Principe

Toute la thermodynamique d'un système peut être déduite de sa somme d'états. Malheureusement, dans le cas d'un système avec interactions (gaz réel, phase condensée), il n'est pas possible d'exprimer cette somme d'état sous la forme d'une expression analytique. L'étude de systèmes représentatifs de la réalité reste cependant possible grâce aux simulations numériques. On se propose ici d'étudier N particules (molécules monoatomiques) en interaction dans un espace à deux dimensions (besoin de moins de particules par rapport à 3 dimensions : le calcul est plus rapide).

Nous utilisons ici une méthode de simulation basée sur la statistique : la méthode de *Monte Carlo* et plus particulièrement l'algorithme de Metropolis, qui fut l'une des toutes premières applications des ordinateurs (l'article fondateur date de 1949!). Le formalisme et les démonstrations mathématiques sont hors programme et ne sont pas nécessaires ici. On se limite à une explication succincte basée sur la distribution de Maxwell-Boltzmann :

- Un système est décrit par un micro-état (ensemble des positions atomiques), et possède une énergie totale U
- Si on perturbe le système (par exemple : on fait bouger une molécule), son énergie change de dU
- Si $dU < 0$, le nouveau système est plus stable : il sera forcément plus représentatif de la réalité
- Si $dU > 0$, le nouveau système, moins stable, a une probabilité $\exp(-\frac{dU}{kT})$ d'être tout de même représentatif, car la température autorise des barrières d'énergie à être franchies.

Si on fait suffisamment de perturbations, on montre qu'on finit par converger vers des micro-états qui sont tous représentatifs d'un même macro-état, donc du système réel correspondant ! On peut alors analyser l'ensemble des micro-états visités (appelés ici *configurations*) pour en tirer des moyennes directement comparables à l'expérience.

2 Algorithme

Un *cycle* est un ensemble de N mouvements de molécules. Chaque mouvement est accepté ou refusé suivant le critère énergétique vu dans la section précédente. Une simulation est une suite de N_C cycles. On peut intégrer directement les boucles et tests en python dans notre pseudo-code :

```
<generer une configuration initiale (N particules) et calculer son energie>
for ic in range(NC):
    for i in range(N):
        <faire bouger la particule i>
        <calculer dU causee par le mouvement>
        if dU<0:
            <accepter le mouvement>
        elif uniform(0,1)<exp(-dU/kT):
            <accepter le mouvement>
        else:
            <refuser le mouvement>
    <faire des analyses sur la configuration ici>
```

Notez le test d'acceptation lorsque $dU > 0$: il s'agit du tir d'un événement basé sur sa probabilité (vu dans le dernier chapitre du cours).

Les actions entre $<>$ doivent ensuite être détaillées. Par exemple, on générera la configuration initiale en créant N positions (x, y) dans un espace donné (appelé la *boîte de simulation*) :

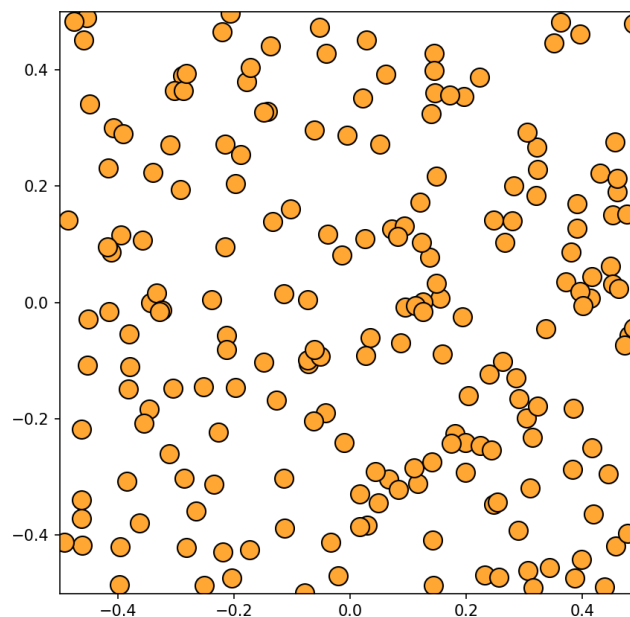


FIGURE 1 – Configuration initiale générée aléatoirement : 150 particules sont placées dans un carré de côté 1, centré sur l'origine.

Le calcul de l'énergie nécessite d'explorer toutes les paires de particules, et d'additionner toutes les contributions qui seront ici fonction de la distance r entre les deux particules, par exemple :

$$u(r) = \frac{A}{r^2}$$

ou bien quelque chose de plus réaliste (voir l'exercice 4 du TD 3). On utilisera dans ce projet les conditions aux limites périodiques, qui permettent d'éviter des effets de bords, car les molécules dans les coins de la boîte doivent avoir le même environnement que les autres (vu en cours⁴). Pour calculer dU , il suffit de calculer seulement les contributions à l'énergie de la particule i qu'on déplace.

Accepter un mouvement revient à valider la nouvelle position, donc de la mettre à jour dans notre tableau de coordonnées. Si on refuse le mouvement, on ne fait rien. Les analyses se limiteront au début à la création d'un visuel comme ceux montés ici afin de vérifier que les positions des molécules évoluent bien vers quelque chose d'organisé. En effet, les molécules trop proches créent une énergie très élevée, situation qui doit être corrigée par l'algorithme. En fonction de la température imposée, on pourra alors converger vers des systèmes assez « mobiles » rappelant un liquide, ou même une organisation plus statique rappelant un solide :

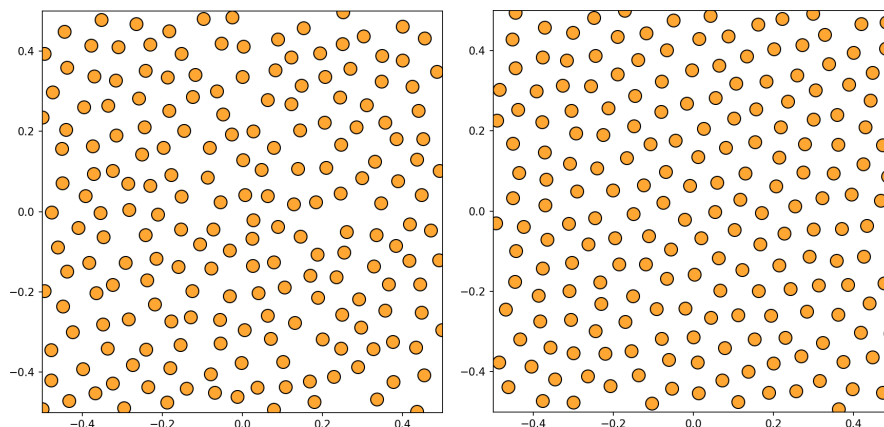


FIGURE 2 – Exemple de configurations après une équilibration de plusieurs cycles. À gauche : structuration légèrement visible rappelant un liquide ; à droite : forte structuration, proche d'un solide (T plus basse).

3 Déroulement du projet

Vous devrez organiser rigoureusement le travail avec des fonctions qui effectuent les opérations élémentaires nécessaires au modèle. Commencez par inventorier les variables nécessaires pour construire un système initial. Vous devez aussi penser aux unités des différentes grandeurs : il est conseillé de travailler en *unités réduites*, avec une boîte de dimension 1 et des énergies

4. Sur chaque dimension, la distance corrigée est $r' = r - \text{int}(r)$ si la longueur de la boîte est de 1.

arbitraires. Le choix de la température (on prend $k = 1$) se fera alors en fonction.

Ensuite, listez chaque action élémentaire pour en faire une fonction, en augmentant graduellement la complexité de l'action effectuée : calcul d'une distance, puis calcul de l'énergie d'une paire, puis calcul de l'énergie d'une molécule avec toutes les autres, puis mouvement d'une molécule, puis enfin effectuer un cycle complet. Le programme principal n'en sera que plus simple et plus lisible.

Il faut rapidement visualiser votre système, en utilisant un graphique matplotlib très simple comme les images montrées ici. Vous pouvez aussi afficher l'énergie totale pour chaque cycle afin de vérifier qu'elle diminue bien.

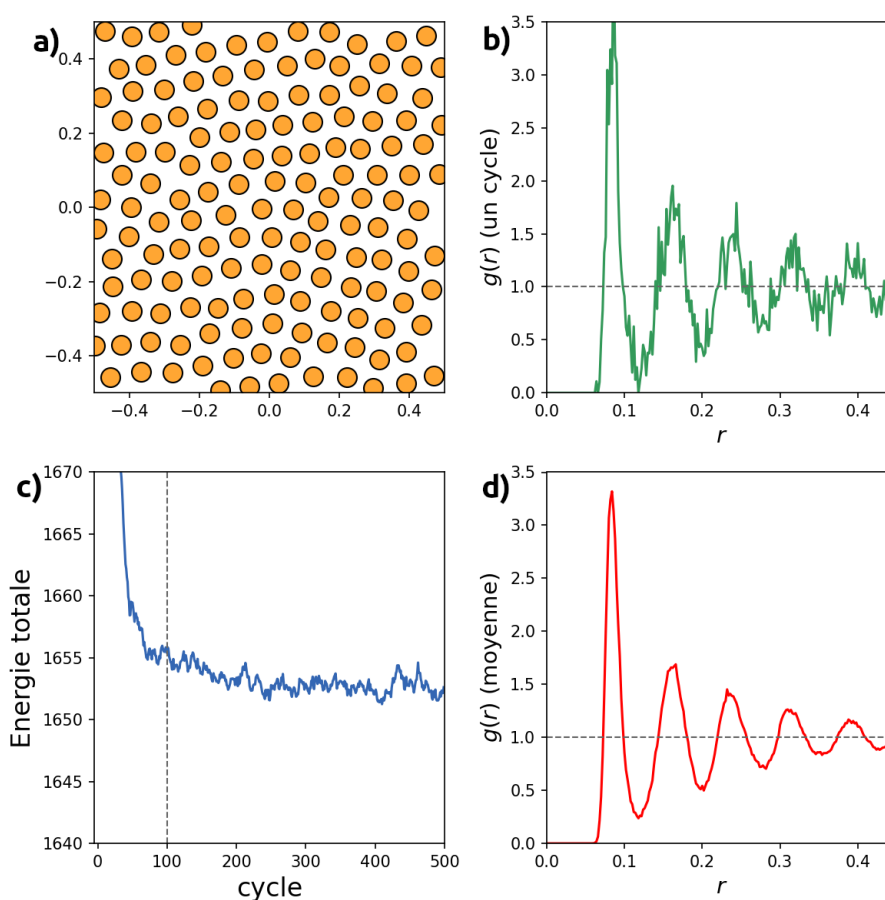


FIGURE 3 – Synthèse d'une simulation : a) visuel de la configuration ; b) fonction de distribution radiale calculée pour la configuration ; c) évolution de l'énergie totale ; d) fonction de distribution radiale moyenne calculée sur tous les cycles.

La structure du système peut être étudiée en calculant la *fonction de distribution radiale* $g(r)$ qui peut être ensuite moyennée sur toutes les configurations (représentatives) explorées. Il s'agit du rapport entre le nombre de paires de molécules trouvées à une distance r et le nombre de ces paires qu'on aurait s'il n'y avait aucune structure. Le numérateur est calculé en calculant toutes les distances (comme pour l'énergie). Le dénominateur s'obtient en considérant la densité

moyenne de la boîte. L'aspect de $g(r)$ change en fonction de la température, et permet de capturer l'ensemble de la structure d'un système. Demandez plus de précisions à votre enseignant si vous voulez calculer $g(r)$.

Vous pouvez affiner votre simulation en adaptant automatiquement le déplacement des molécules dans le but d'imposer non pas l'amplitude du déplacement mais la proportion de mouvements acceptés. Cela permet d'optimiser l'efficacité de votre code.

Vous pouvez aussi choisir de travailler en unités réelles, ou utiliser un potentiel d'interaction réaliste. Vous pourriez alors représenter des séparation de phase. Il serait théoriquement possible de simuler des molécules polyatomiques, mais cela nécessiterait d'ajouter des mouvements de rotation/vibration plus compliqués et hors programme.